Digital Research Toolkit for Linguists

Week 15: Version control, course outro

Anna Prysłopska

July 15, 2025

Psycholinguistics and Cognitive Modeling Lab

Git warning

`warning: in the working copy of FILE LF will be replaced by CRLF the next time Git touches it.`

"In the working copy of FILE, the Unix end of line markers will be replaced by the Windows end of line markers the next time Git touches it."

LF (Line Feed) used by Linux and macOS `\n`

CRLF (Carriage Return + Line Feed) used by Windows `\r\n`

$\rightarrow$ good question for AI

Questions?

Data → Understand → Communicate → Share

Data

create, find, data types, formats, encoding

Understand

R, packages, import, clean, transform, debug, visualize, export

Communicate

document, create clean and beautiful reports

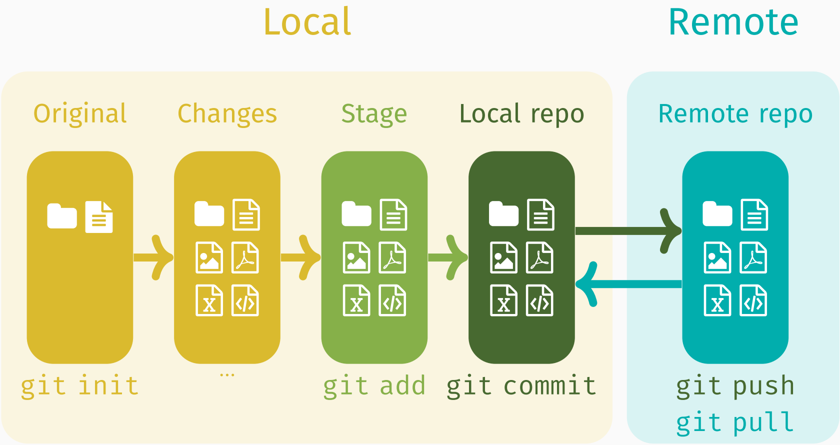
Share

connect, collaborate, backup, replicate

Table of contents

Git + GitHub

Workflow


Local
Remote
Original
Changes
Stage
Local repo
Remote repo
git init
...
git add
git commit
git push
git pull


git commit
AIRBUS A350-1000
1000
A350
git push
AIRBUS A350-1000
1000
A350
git add .

Your history w/ Git

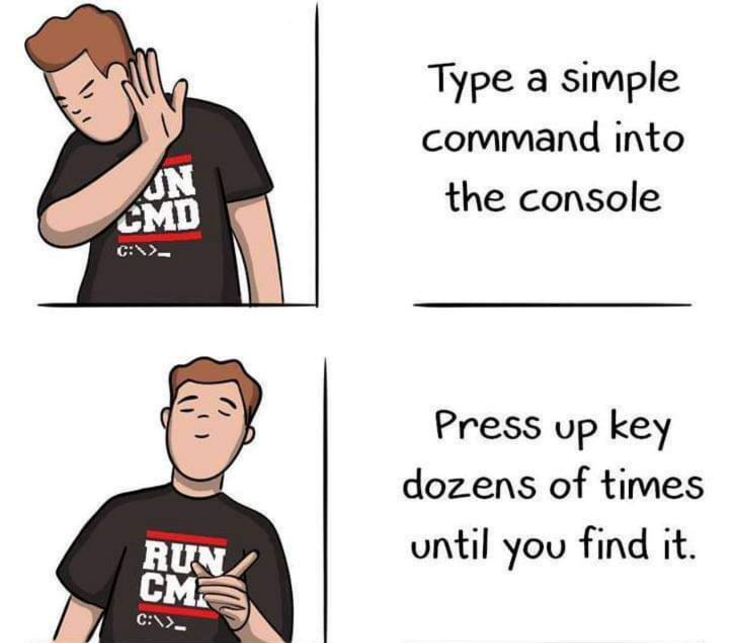
1. Exercise: Create/log into GitHub
2. Exercise: Create a readme.md file
3. Exercise: Create a git repository `git init`
4. Exercise: Stage files `git add readme.md`
5. Exercise: Commit changes `git commit -m "First commit"`
6. Exercise: Make an SSH key pair & add to GitHub
7. Exercise: Create a new repo on GitHub
8. Exercise: Add me as a collaborator
9. Exercise: Make three more commits
10. Exercise: Post your GitHub repo link on Ilias: “Questions and discussion week 15”

Bored? Practice the ssh and the command line: `https://overthewire.org/wargames/bandit/bandit0.html`

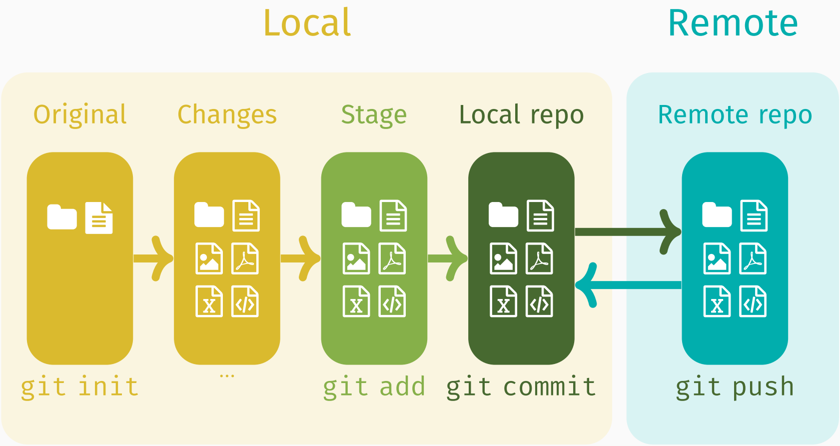
Exercise 11: Push the changes

```
git remote add origin…
git branch -M main
git push -u origin main
```

Command line



Workflow


Local
Remote
Original
Changes
Stage
Local repo
Remote repo
git init
...
git add
git commit
git push

Glossary: Setup

Repository/repo a place where something is stored and managed.

Server the computer storing the repository.

Client the computer connecting to the repository.

Working set/copy local directory/place where you make changes.

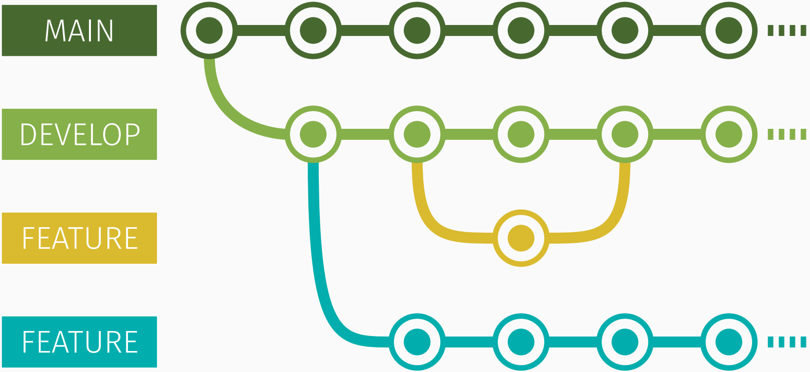
Trunk/main/master the main location for the data in the repository.

Fork is a copy of a repository that you can change to your liking.

Branch a parallel version of the repository at this time.

Merge combine a (feature) branch with the main branch/trunk.

Tree structure



Glossary: Actions (must know)

- ✔`add` tell git to track untracked files.
- ✔`commit` change to tracked files. Has a unique ID ($\approx$ save as) and keeps record of changes. Usually contains a message, i.e. description of changes.
- ✔`status` what are the changes since the last commit?
- ✔`push` send changes to a remote repository.
- ✔`pull` get changes from a remote repository.

Glossary: Actions

`log` list of all changes.

`branch` (make a) parallel version of a repository.

✔`diff` difference in changes between files.

`merge` combining changes from two branches.

`reset` undo/rollback the last commit (move backwards).

✔`revert` undo/cancel a change but make a new commit (move forwards).

Glossary

`git clone URL`	make a new copy of a repository.
`git add file.txt`	tell git to track untracked files.
`git commit -m "msg"`	snapshot of tracked files + message
`git status`	what are the changes since the last commit?
`git push`	send changes to a remote repository.
`git pull`	get changes from a remote repository.
`git diff file.txt`	difference in changes between files.
`git log`	show the commits + IDs + messages.

which program (topic) do what more details

Commit often

Make small commits often to make tracking changes and finding mistakes easy.

Review changes before committing

Use `git diff` to check what changes were made to the files you're committing.

Meaningful messages

Write clear and concise commit messages.

✖ "Add files"

✔ "Added participant information to the methods section"

Avoid committing large or personal files

Exclude unnecessary files and sensitive information. Keep it minimal and use `.gitignore`. GitHub will reject files that are too large.

Branch out

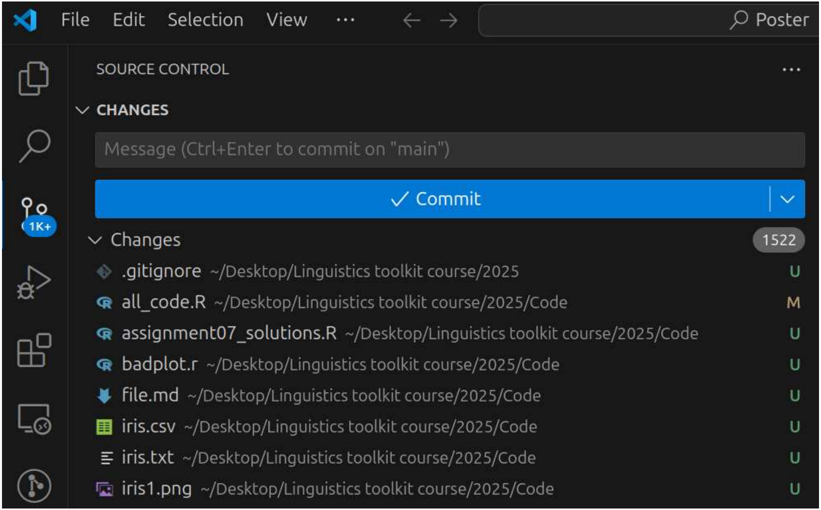
Use branches when collaborating, adding new features, fixing bugs, and experimenting. Keep the main branch stable.

Documentation

Keep the documentation up to date (`readme.md`), include information about setting up, dependencies, examples, contact information, known issues etc.

Others: regularly review code, protect branches, track issues, organize milestones, document in a wiki

Git in VS Code

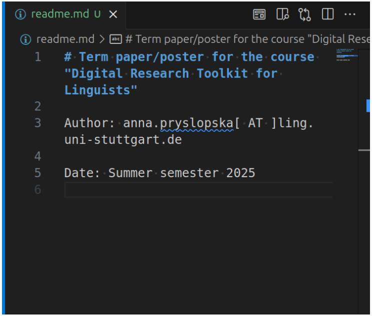


Restoring and reverting

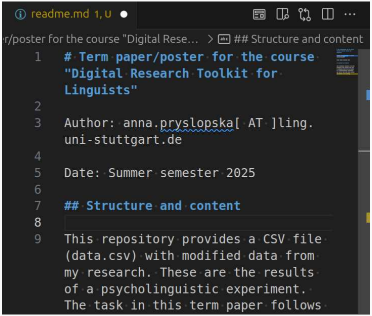
Log

`git log`	full commit history
`git log --oneline`	short hash + message
`git shortlog`	author + messages
`git log --oneline --graph --all --decorate`	compact, visual tree
`gitk`	GUI

Original



Changed



Going back in time

*You have added FILENAME but **not commited** the changes. Now you...*

... want to unstage this file because it does not fit the current commit.

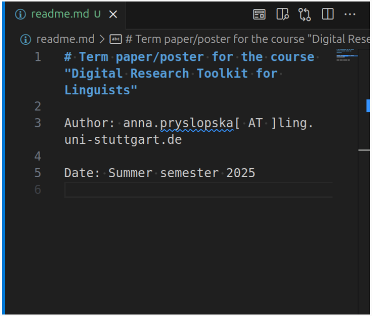
```
git restore --staged FILENAME
```

*... **completely undo all changes** in the file and restore to a previous commit state:*

```
git checkout FILENAME
```

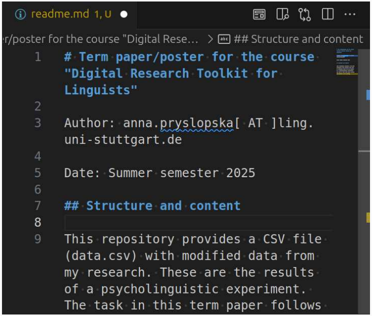
Restore vs checkout

Original



after `checkout`

Changed



after `restore`

Going back in time

*You want to undo a change that has been **commited but not pushed***

`git log --oneline`	check the commit ID, e.g. `3add00e`
`git reset ID`	undo **until** the commit (move backwards) but **does not change** the file
`git reset --soft ID`	undo **until** the commit (move backwards) but **does not change the file** and **keeps the staging**
`git reset --hard ID`	same & **changes** the file to earlier version
`git reset --soft HEAD~`	reset the last commit

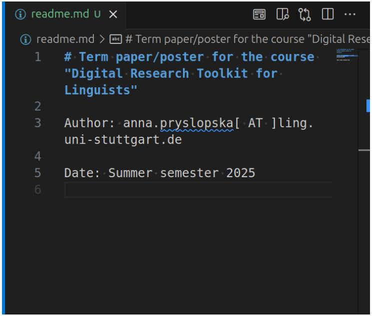
Going back in time

You want to undo a change that has been ***commited and pushed***

`git revert ID` — undo **only this** change completely and move forwards as **new commit**

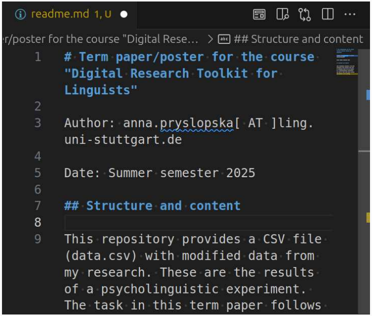
Reset

Original



after `reset --hard`

Changed



after `reset ID`

after `reset --soft`

Restore, checkout, reset, revert

Command	Effect on file	Effect on repo
✅ `restore`	keep changes	unstage
✅ `revert`	delete changes	keep commit, make a new commit
⚠️ `reset`	keep changes	abandon commit, go to earlier commit
⚠️ `reset --soft`	keep changes	abandon commit, go to earlier commit
⚠️ `reset --hard`	delete changes	abandon commit, go to earlier commit
⚠️ `checkout`	delete changes	unstage, go to earlier commit

HEAD commit

`d8c6f90`	HEAD Removed everything	newest
`4581f94`	Drafted the conclusions	← I want to go back here
`57ff630`	Summarized the results	
`66528ea`	Added the experiments	
`a71ea1c`	Wrote the introduction	oldest

Reset HEAD 🗣️

```
git reset --hard 4581f94
```

← Disappeared without a trace

`4581f94` HEAD Drafted the conclusions — newest

`57ff630` Summarized the results

`66528ea` Added the experiments

`a71ea1c` Wrote the introduction — oldest

Revert HEAD 🗣

`git revert HEAD -m "Added everything back"`

`g4c5t4g` HEAD Added everything back		newest
`d8c6f90` Removed everything		← old HEAD
`4581f94` Drafted the conclusions		
`57ff630` Summarized the results		
`66528ea` Added the experiments		
`a71ea1c` Wrote the introduction		oldest

Course wrap-up

Review

Data → Understand → Communicate → Share

Data: create, find, data types, formats, encoding

Understand: R, packages, import, clean, transform, debug, visualize, export

Communicate: document, create clean and beautiful reports

Share: connect, collaborate, backup, replicate

Data and understanding data

- ✔ R and RStudio IDE
- ✔ Packages (installing and loading)
- ✔ Working directory
- ✔ Directories and file hierarchy
- ✔ Scripts
- ✔ Assignment
- ✔ Scripts
- ✔ Data types
- ✔ Encoding
- ✔ Reading in data

Understanding data 1/2

- ✔ Inspecting data
- ✔ Renaming
- ✔ Selecting
- ✔ Dealing with missing data
- ✔ Coding basics
- ✔ Logic
- ✔ Filtering
- ✔ Arranging
- ✔ Pipes

Understanding data 2/2

- ✔ Tidy code
- ✔ Power of names
- ✔ Joining data frames
- ✔ If…else
- ✔ Grouping
- ✔ Summarizing
- ✔ Getting help
- ✔ Data visualization goals
- ✔ Accessibility and WCOG
- ✔ Plot types and choice of visualization
- ✔ Plotting in R with `ggplot2` and `esquisse`

Communicating results 1/2

- ✔ Documentation and why it's important
- ✔ Markdown and Pandoc
- ✔ Creating documentation with Quarto
- ✔ Knitting to HTML from R
- ✔ Scientific document structure
- ✔ Creating tables
- ✔ Including plots
- ✔ Cross-referencing and hyperlinking

Communicating results 2/2

- ✔ Large project management
- ✔ Text editors and their uses
- ✔ Basic Bib(La)TeX and citation
- ✔ Reference managers
- ✔ Literature research
- ✔ AI for the humanities and research

Sharing and collaborating

- ✔ Command line, terminal, console, and shell
- ✔ Version control
- ✔ Git basics
- ✔ GitHub
- ✔ SSH

Questions?


MY EXAM DAY
morning
exam starts
exam ends
I passed
I failed
WeKnowMemes